

VCF Core: A Semantic Foundation for RDF Representations of VCF

Elias Crum^{1,2,*}, Bart Buelens², Gökhan Ertaylan² and Ruben Taelman¹

¹IDLab, Department of Electronics and Information Systems, Ghent University – imec, Belgium

²Flemish institute for Technological Research (VITO) Mol, Belgium

Abstract

Medical applications of genomic data depend on connecting variant observations to clinical information and biomedical knowledge. Linked Data offers a framework for this integration, but the Variant Call Format (VCF) does not natively provide an RDF representation of the observations and their interpretation context. We present VCF Core Vocabulary 2.1.2, a shared semantic target for representing VCF files, header declarations, records, annotations and sample data. The vocabulary integrates existing provenance and genomic models while retaining VCF-specific field definitions, ordering and missingness. Expanded sample calls expose individual values, whereas condensed, sample-ordered vectors provide an alternative for cohort data. Version-specific registries and validation profiles address VCF 4.1–4.5, including 122 reserved INFO and FORMAT declarations for VCF 4.5. A curated assessment records representation evidence for all 104 inventoried logical-model constructs and enforcement evidence for 87, and a set of 123 curated alignments documents connections to existing variation models and reference ontologies. We evaluate the vocabulary with executed checks rather than term counts. A specification-derived assessment of 94 requirements over VCF 4.1–4.5 supplies queries and expected answers authored in advance. Replaying all of it over graphs from a second, separately implemented producer — every fixture, every case that has a test — gives 840 checks, 693 passing for both producers. The 102 checks where they differ span 15 requirements, and examining each shows that none arises from the vocabulary being unable to express something: every absent property is one the release declares and the converter does not yet write. A semantic round-trip recovers the fixed columns of all 109 records and all 151 sample genotypes, in the expanded profile, from structured properties alone, and one integration example answers an allele-dependent question under VCF 4.5 local alleles with checked answers and recovered provenance. The replay also found a silent correctness defect in local-allele handling that per-producer testing had not, which we report as evidence for keeping representational mechanism, implementation and correctness apart. We then discuss how this foundation could support reusable conversion engines, traceable links to biomedical resources, and SPARQL queries combining genomic observations with medically relevant information.

Keywords

Variant Call Format, RDF, Genomic data, Vocabulary, Data integration, SHACL

1. Introduction

Healthcare research and personalized medicine depend on connecting clinical observations, molecular data, and biomedical knowledge. These resources are distributed across institutions and databases, where differences in data structures, identifiers and terminology complicate their combined use [1, 2]. Semantic Web approaches provide a basis for addressing this heterogeneity through shared identifiers, explicit relationships and reusable vocabularies [3, 4]. Leveraging these strategies has enabled harmonizing clinical data across institutions while integrating knowledge about genes, diseases, and phenotypes to support biomedical investigations [1, 2]. Meaningful integration of these distributed resources nevertheless depends on retaining the context in which source observations were recorded, not only on adopting common identifiers or terminology [1]. For genomic data, this motivates a reusable semantic representation that preserves source observations and their interpretation context while enabling

SWAT4HCLS 2027, February 1–4, 2027, Basel, Switzerland

*Corresponding author.

✉ elias.crum@ugent.be (E. Crum); bart.buelens@vito.be (B. Buelens); gokhan.ertaylan@vito.be (G. Ertaylan); ruben.taelman@ugent.be (R. Taelman)

🆔 0009-0005-3991-754X (E. Crum); 0000-0001-7734-3747 (B. Buelens); 0000-0001-5602-6435 (G. Ertaylan); 0000-0001-5118-256X (R. Taelman)



© 2027 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

connections to clinical information and biomedical knowledge.

The Variant Call Format (VCF) is widely used to exchange genomic variant data [5]. A VCF file can be understood as a structured table in which each data row describes a genomic location and the sequence observed there relative to a reference genome. For each row, a record contains the reference allele, one or more alternative alleles, and optional annotations describing properties such as quality, filtering, or predicted biological effects. When individual samples are included, additional columns record sample-level observations such as the genotype or sequencing depth. A header at the beginning of the file defines many of these fields and specifies how their values should be interpreted [6].

These relationships are important because VCF often encodes meaning indirectly through the structure of the file. For example, the reference allele is assigned index 0 and alternative alleles are numbered in the order in which they appear. A genotype written as 0/2 therefore does not itself identify two DNA sequences: it indicates that the sample carries the reference allele and the second listed alternative allele. Similarly, the meaning and expected number of values for an annotation may depend on a definition given in the file header. This simple example illustrates a broader characteristic of VCF: the format achieves compactness by relying on relationships that are understood within the file rather than stated explicitly with each value. More complex cases, discussed later, depend on several such relationships at once. When VCF observations are integrated with information outside the file, these implicit relationships must remain available if the observations are to be integrated meaningfully with other genomic, clinical, and biomedical data. The challenge is therefore not that VCF lacks contextual information, but that much of this context is encoded through the internal structure of the file rather than exposed as explicit, independently addressable relationships.

A semantic representation offers a way to make these dependencies explicit while preserving their connection to the original observations. Doing so supports semantic interoperability and reuse, central concerns of the Findable, Accessible, Interoperable and Reusable (FAIR) principles [7]. The wider FAIRness of a dataset also depends on its identifiers, metadata, access conditions and provenance, rather than on the choice of file format alone [8]. The Resource Description Framework (RDF) provides a graph-based model of resources and their relationships, while Linked Data principles support the use of shared identifiers and links across datasets [9, 10]. Together, these provide a basis for exposing VCF observations and their interpretation context alongside connections to external biomedical resources, which is necessary for any downstream clinical or research applications [11].

Existing VCF-to-RDF approaches provide important foundations for this work. The Genomic Feature and Variation Ontology (GFVO) represents VCF concepts within a broader model of genomic features and variation, while VCF2RDF presents an isomorphic semantic mapping of the format [12, 13]. HERO-Genomics places VCF files and records within a broader integration model, while the Genome Variation Ontology (GVO) describes variation, including complex structural variants [14, 15]. The GA4GH Variation Representation Specification (VRS) addresses computable variation descriptions and federated identification rather than all source-file details [16]. These complementary purposes motivate a distinction between representing an observation as recorded and linking it to a biological interpretation. Our aim is therefore not simply to establish that VCF can be represented in RDF, but to provide a version-aware representation that combines broad representation of the source format with explicit connections to existing genomic models. Such a target would allow conversion and querying approaches to develop without repeatedly defining application-specific interpretations of VCF structure, as has been done in the past.

1.1. Why version and source context matter

VCF format has been evolving since its first public release in 2011. At the time of writing, the specifications are maintained by the Global Alliance for Genomics and Health (GA4GH) [17] with versions 4.1–4.5 hosted on a GA4GH-maintained github repository [18]. The most current specification is VCF v4.5 and was released February 25, 2026 [6]. VCF 4.1–4.5 share the basic structures described above but differ in supported declarations and constraints [19, 6]. For example, later versions introduce new ways to describe variants and their associated metadata, while also refining how existing fields should be

interpreted. To further complicate format versioning, the observations recorded in a VCF must remain distinguishable from the biological entities they describe. Two files may report the same alteration with different annotations or quality estimates, just as a sample column may refer to a patient specimen, a pooled sample, or a synthetic example. A common representation should therefore preserve both version-specific semantics and the provenance of each observation, without automatically equating VCF records with normalized variants or sample columns with patients.

Here, we present the VCF Core Vocabulary (*vcfcv*) [20]. The contribution combines source-context preservation, expanded and condensed sample representations, documented alignments, and version-specific artifacts for VCF 4.1–4.5. We assess these capabilities through specification-derived queries with expected answers, a replay across separately implemented producers, semantic round-trip checks and an allele-dependent biomedical integration example with recovered provenance. These assessments distinguish the existence of a representational mechanism from its implementation and the correctness of the resulting data. We then discuss how this foundation could support reusable conversion engines and integrated, medically relevant genomic querying. Throughout, *vcfc* denotes <https://w3id.org/vcf-core/vocab#>.

2. Methods

2.1. Assessment objectives, evaluated artifacts, and essential model

The assessment undertaken in this study addressed three questions. **RQ1:** Does VCF Core preserve the intended meaning of the VCF constructs it claims to support? **RQ2:** Do the version-specific artifacts and alternative sample profiles behave as documented? **RQ3:** Does the resulting representation support a reproducible integration task while retaining source context? **RQ4:** How does VCF Core relate to other existing VCF semantic models?

We address RQ1 and RQ2 by testing the vocabulary against requirements drawn from the VCF specifications followed by checking whether the same expected results are obtained from converter-generated RDF. A separate biomedically-motivated integration case study addresses RQ3. To address RQ4, we conduct a comparative representation assessment of existing semantic models that model VCF contents and identify how they relate to the terms provided by VCF Core.

All methods described below, code used to execute them, and environments necessary to reproduce them are available in the git repository https://github.com/ecrum19/SWAT4HCLS_2027.

2.2. Extracting VCF constructs, rules, and meanings into the VCF Core Vocabulary and Accompanying SHACL rule-sets (RQ1)

VCF Core was developed iteratively from the earlier VCF-RDFizer vocabulary [21]. Successive revisions combined Claude Opus 5 and GPT-6 Astra assistance with expert human review to extract VCF constructs, field meanings, rules and representation constraints from the VCF 4.1–4.5 specifications [19, 6]. The extracted interpretations informed vocabulary terms and relationships, version-specific field definitions, and accompanying SHACL constraints; human review guided their acceptance and revision. Two complementary assessments tracked this development: an initial empirical inventory of VCF 4.5 constructs, followed by a specification-derived assessment of information requirements across VCF 4.1–4.5 [19].

Initial empirical inventory. The first assessment, which helped guide VCF Core vocabulary development, is available for reproducibility in the git repository directory `RQ1/vcf45-inventory/`. The inventory organized 104 curated logical-model constructs by specification area and linked each to vocabulary terms and available evidence. The evidence distinguished materialized example graphs, SHACL shapes or decoded checks, regression probes that reject violating graphs, and executed queries with fixed expected answers. For each construct, human and agentic reviewers recorded three separate judgements: whether its information was *preserved*, whether it was *structured* for retrieval through

graph patterns without parsing a literal, and whether a validation rule *enforced* a stated requirement. Representation was classified as *full* when both preservation and structure held, *partial* when only preservation held, and *absent* otherwise; enforcement earned no additional representation credit. The reporting script [RQ1/vcf45-inventory/report.py] recomputed these classifications and totals, checked cited vocabulary terms and query paths, and incorporated recorded validation results. Byte-level serialization checks were reported separately from logical-model coverage.

Specification extraction and requirement registration. A more complete assessment of vocabulary completeness, recorded in RQ1/methodology/, began with local copies of the VCF 4.1–4.5 LaTeX specifications, identified by source URLs and verified against SHA-256 digests in RQ1/methodology/sources.lock.json. A bounded text reader partitioned each complete source into section intervals, retaining tables, examples and changelogs for review, and separately extracted explicit reserved INFO/FORMAT Number/Type declarations. Semantic interpretation remained an authored, reviewed step: passage-level assertions recorded source line ranges, requirement links, exclusions, ambiguities and remaining test gaps in RQ1/methodology/inputs/source-assertions.json. The requirement register recorded stable identifiers, retrieval questions, applicable versions, interpretations and hashed source anchors; missing capabilities identified during the source audit expanded the register from 50 to 94 requirements. The scope covered source identity, field meaning, version context, ordering and missingness. Binary encoding, compression, indexing, producer estimation algorithms and general invalid-input rejection were excluded from this retrieval assessment.

Human review of interpretations and test evidence. The review record documents two preparatory agent passes followed by human review and acceptance by author Elias Crum. Review examined source meaning, version applicability, proposed RDF patterns, expected answers, and the adequacy of the linked cases. This process distinguished normative rules from inconsistent examples, recorded reasoned corrections, and identified cases that required reviewer-authored fixtures. For example, a local-allele test whose indices made the mapping trivial was supplemented with a non-identity allele subset; inconsistent phase-set examples were corrected with the rationale recorded. Where the source supplied no defensible expected answer, the limitation remained explicit. RQ1/methodology/REVIEW-REPORT.md and RQ1/methodology/inputs/review.json preserve these decisions, including reviewer attribution, dates, rationales, and evidence fingerprints; agent-authored interpretations are identified separately from human acceptance.

Executable retrieval tests. Each case linked a registered requirement and VCF version to a fixture, a SPARQL query and expected answer rows authored from the source interpretation and fixture before query execution. Reused fixtures retained origin records; reviewer corrections, and newly authored examples were documented in the review evidence. The vocabulary repository's Python fixture materializer generated RDF in expanded and condensed profiles, which the assessment queried with RDFLib Preservation tests could explicitly decode compound VCF literals, whereas structure tests required access through RDF properties and indices. The runner compared answer rows including their multiplicities, rejected queries that returned the expected answer on an empty graph, and required every passing query to change its answer when at one or more referenced VCF Core predicate(s) was deleted. It also checked that emitted vocabulary terms were declared and that object and datatype properties had the appropriate resource or literal values. These controls tested data dependence and representation consistency; they did not establish unrestricted conversion correctness or comprehensive SHACL enforcement.

Coverage and progress measures. Requirements were scored separately for each applicable VCF version, sample profile, and retrieval axis. A requirement was *demonstrated* only when all its supplied tests passed, *partial* when some passed, *not demonstrated* when tested but none passed, and *unassessed* when no test was supplied. The numerator counted demonstrated requirements with equal weight;

the denominator retained all applicable registered requirements, including unassessed ones, without fractional credit. Alongside these scores, the source audit exposed assertions with targeted cases, partial evidence, or no targeted case, since a passing requirement-level example need not exercise every linked assertion. Comparison of extracted Number/Type declarations with the versioned RDF registries was an additional consistency check, limited to explicit declarations rather than all field meanings. The two assessments therefore measured different populations and their percentages were not combined; neither established complete specification coverage.

Reproducible assessment artifacts. Pinned specifications and authored assertions, requirements, fixtures, queries and expected answers were kept separate from generated reports and RDF witnesses [20]. With the repository dependencies installed, the specification-derived assessment runs offline: `npm run methodology:build` regenerates the evidence, `npm run methodology:check` recomputes it without writing and detects stale outputs, and `npm run methodology:test` exercises the assessment's safeguards. Adding `-require-reviewed` to the check command also requires current recorded acceptance of the requirements and source audit. Provenance records input hashes and the RDFLib version; changes to substantive hashed evidence invalidate requirement acceptance, while release-version metadata alone is normalized before hashing. For the initial inventory, `npm run coverage:report` runs the separate byte checks and regenerates the report; its summary of SHACL fixture results reads the recorded validation output, while `npm run validate:force` reruns the complete validation suite.

VCF Core retains a file, its header declarations, records, and sample observations as distinct resources (Figure ??). A `VariantCall` groups the observations associated with a record rather than identifying an additional source-file line. File-scoped identifiers and provenance distinguish observations even when they concern the same alteration. A `VCFSample` identifies a source column, not automatically a patient or specimen. Direct reuse of `PROV` and `DCAT` supports source descriptions, while `SO` and `FALDO` provide connections to alteration and location models [22, 23, 24, 20]. These connections do not perform variant normalization or establish instance identity across files.

Field definitions are separate from their occurrence in a file header, and values can link to the declaration needed to interpret them. Lexical values, typed accessors and explicit missingness remain distinguishable; ordered allele resources retain the relationship between allele indices and sequence alternatives. These mechanisms are the objects of assessment, rather than evidence of correct conversion by themselves.

Evaluated artifacts. The assessment used VCF Core 2.1.2 together with its accompanying vocabulary files, version-specific definitions, validation rules, mappings and test data [20].

Every check was run twice, once per producer, and both results were recorded. The first run used RDF built from the test fixtures by the vocabulary repository's own Python materializer. The second used RDF produced from the same fixtures by VCF-RDFizer 3.0.3 [25], which is an unrelated implementation: it converts through RML mappings executed by RMLStreamer rather than by constructing triples in Python. Naming both producers matters, because a check passing under one says nothing about the other, and the difference between them is the object of study in Section 3.3.

The second run covers the whole assessment, not a subset. Each of the 38 fixtures the assessment refers to is converted in both sample profiles, so every case that has a test is replayed and every requirement it exercises is compared. The fixture set is not written down in the reproduction script; it is derived at run time from the assessment's own case register, so a fixture added to the assessment is replayed automatically rather than being silently left out. The full outcome of every check is recorded — both producers' results for every case, version, profile and retrieval axis, not only the cases where they differ — so a reader can confirm the reported totals rather than take them on trust. An earlier version of this evaluation replayed ten of the 38 fixtures; that narrower comparison is superseded and is not reported here. The producers share an author but not implementation code; this is an implementation cross-check, not independent third-party reproduction.

The evaluated converter release. Every figure reported here comes from VCF-RDFizer 3.0.3 (be658a2), run from the published image pinned by digest sha256:31f1361b. Requirements that diverge between the two producers are listed individually with their per-check outcomes in the run's result files, so no total has to be inferred from another.

Reproducing all results. One command regenerates all experimental data in this paper:

```
sh reproduce/run.sh
```

It fetches the vocabulary at v2.1.2 and both converter releases at their tags, pulls the two container images by digest, converts the ten fixtures in both sample profiles with each release, runs every analysis, and writes one self-contained run directory. That directory holds the machine description and toolchain versions (`env.json`), every command executed with its exit code and duration (`steps.jsonl`), the per-step output (`logs/`), the converted graphs each result was computed from (`graphs/`), the result files (`results/`) and a summary. The run reported here took 616 seconds over 15 steps with no failures.

Provenance that a reader would otherwise have to reconstruct is recorded by the run itself: the pinned specification revisions with their SHA-256 digests, the fixture set, the SPARQL engine that executes every coverage, round-trip and integration query (rdflib 7.6.0), the SHACL validator (pySHACL) with the ontology graph loaded and RDFS inference enabled, and the container image digests. The SHACL configuration is not incidental — without the ontology graph and RDFS inference, `sh:class` constraints cannot see the vocabulary's subclass axioms and conforming data reports violations, so the validator is run the way the vocabulary's own test suite runs it.

Representation needed for the assessment. VCF Core retains a file, its header declarations, records and sample observations as distinct resources (Figure ??). A `VariantCall` groups the observations associated with a record rather than identifying an additional source-file line. File-scoped identifiers and provenance distinguish observations even when they concern the same alteration. A `VCFSample` identifies a source column, not automatically a patient or specimen. Direct reuse of PROV and DCAT supports source descriptions, while SO and FALDO provide connections to alteration and location models [22, 23, 24, 20]. These connections do not perform variant normalization or establish instance identity across files.

Field definitions are separate from their occurrence in a file header, and values can link to the declaration needed to interpret them. Lexical values, typed accessors and explicit missingness remain distinguishable; ordered allele resources retain the relationship between allele indices and sequence alternatives. These mechanisms are the objects of assessment, rather than evidence of correct conversion by themselves.

The expanded profile exposes individual sample calls and FORMAT values through graph patterns. The condensed profile stores sample-ordered vectors with an explicit definition, encoding and payload; the `VCFTextVector` encoding retains tab-separated values in sample order. Both retain file, header and record context, but accessing an individual condensed cell requires decoding. VCF 4.1–4.5 share the model while selecting version-specific registries and validation overlays from the file's declared version [19, 6, 20]. This separates representation choices from the version-dependent rules tested below.

3. Specification coverage and implementation evidence

3.1. One representation with version-specific rules

VCF Core uses one shared model for common structures and version-specific registries and validation overlays for VCF 4.1–4.5 [20]. The registries contain 40, 40, 67, 78 and 122 explicit reserved declarations, respectively. For VCF 4.5 these comprise 51 INFO and 71 FORMAT definitions, including three entries for parameterized base-modification key families. Prose-only fields in older versions are not assigned Number or Type constraints absent from their specifications.

The overlays condition rules on the owning file’s declared version. Thus, the CIPOS example in Section ?? can be assessed under the appropriate historical or current rule, while the LA, LR, LG and M cardinalities are restricted to VCF 4.5. Shared validation is divided into portable SHACL structure checks, SPARQL-based cross-resource checks and raw/parsed-value consistency checks [26, 20]. This organization permits different validator capabilities without implying that each layer independently validates the entire format.

3.2. Results of the two specification assessments (RQ1)

Coverage claims are easy to overstate because three different things are routinely reported as one number. We keep them apart. *A mechanism exists* when a term, rule or documented pattern is available; this establishes nothing about correct instantiation. *An example instantiates it* when a fixture demonstrates the mechanism in use; this establishes nothing about general correctness. *Behaviour is independently checked* when an executed query agrees with an expected answer established separately from the code that produced the graph. Only the third supports a correctness claim, and only within what was tested.

Two assessments and two cross-checks address three questions: whether VCF Core preserves the intended meaning of the constructs it claims to support (RQ1); whether the version-specific artifacts and the two sample profiles behave as documented (RQ2); and whether the result supports a reproducible integration task that retains source context (RQ3).

Initial empirical inventory. The curated VCF 4.5 inventory enumerates 104 logical constructs across file and header organization, declarations, types and cardinalities, lexical rules, fixed fields and alleles, genotypes, structural variation, repeats and gVCF. All 104 were classified as preserved and structured, with no partial or absent representations; 87 (83.7%) additionally had a rule checking a stated requirement [20]. Table 1 gives the distribution by specification area. The assigned evidence tiers were query for 7 constructs, regression probe for 23, shape or decoded check for 62, and materialized fixture for 12. These tiers describe the evidence attached to curated judgements, not 104 independently executed retrieval tests. The remaining 17 constructs had representation evidence without an enforcement rule; this did not reduce their representation classification. The inventory was curated by the vocabulary’s developers, so it maps what its authors listed and cannot reveal an omission nobody listed.

Table 1

Initial VCF 4.5 inventory results by area. Full representation means that both the preserved and structured axes were recorded as satisfied. Enforcement is a separate judgement and does not establish comprehensive validation of a construct.

Specification area	Fully represented	With enforcement
File and header structure	20/20	17/20
Field declaration attributes	8/8	8/8
Type and arity value sets	14/14	14/14
Lexical and encoding rules	7/7	6/7
Fixed fields and allele structure	18/18	18/18
FORMAT and genotype	21/21	16/21
Structural variation, repeats and gVCF	16/16	8/16
Total	104/104	87/104

Reserved declaration agreement. Separately, all 333 reserved Number/Type rows extracted from the specification texts match the versioned registries [20]. Those 333 rows — 31, 31, 69, 79 and 123 per version — are the keys each source declares explicitly in ##INFO=##FORMAT= blocks or reserved-key tables, and are a different population from the registry entries counted above. VCF 4.1 and 4.2 each define a further 29 reserved keys in prose bullets that give no Number and, for INFO, no Type; those rows are inventoried but cannot be compared, so this agreement figure is weighted towards

VCF 4.3–4.5. There were no missing or differing declarations among the compared rows; four rows repeat a key declared elsewhere in the same source, and the three base-modification key families were compared as patterns rather than as every possible concrete identifier. Agreement of declarations did not resolve inconsistencies between a declaration and its prose meaning: for example, the review identified CNL/CNP declarations with Number=G alongside prose defining an ordering by copy number.

Source audit and human-review outcomes. The specification-derived assessment inventoried 344 source-section intervals across VCF 4.1–4.5 and recorded 1,134 authored passage-level assertions. The audit expanded the requirement register from 50 to 94 by adding omitted capabilities; this was an expansion of the assessment scope, not a measured increase in percentage coverage. The review record documents adjudication of 21 ambiguous or defective source passages and acceptance by Elias Crum on 11 September 2026. The generated assessment records no pending requirement reviews, no source sections flagged as needing review, and acceptance of the source audit [20]. Acceptance included explicitly retained limitations; it did not mean that all assertions had executable evidence. Of the 1,134 assertions, 149 had targeted cases, 154 had partial test evidence and 831 had no targeted case. These categories record test presence, not passing outcomes, and repeat information across sections and versions; they therefore form an audit-work inventory rather than an additional coverage denominator.

Review also changed the evidence that could support particular claims. The local-allele cases included a non-identity LAA=2, 4 subset, so a positional mapping could no longer pass merely because local and global indices happened to coincide. Corrected phase-set examples and reviewer-authored phase-quality and PS-versus-PSL examples supplied tests of interpretations for which the printed records were inconsistent or absent. Conversely, legacy GLE (R80, VCF 4.1–4.2) and haplotype-bundle BDP/BCN (R94, VCF 4.1–4.3) remained unassessed because the reviewed sources supplied no defensible serialization and expected-answer basis.

Instantiation and independent checking. The specification-derived assessment reads information requirements from pinned VCF 4.1–4.5 sources and tests them against fixtures. It comprises 94 requirements over 210 cases; each case pairs a SPARQL query with an expected answer authored from the specification text *before* the query was run. The cases exercise 53 distinct requirements in at least one applicable version; the remaining 41 have no registered case in any version. Executing every case on both sample profiles and both retrieval axes yields 840 scored query slots. Of the 210 cases, 165 reuse the same query and expected answer on both axes in both profiles, and the remaining 45 do so in the expanded profile only. Deduplicating identical query, expected-answer and fixture combinations within each case and profile leaves **465 distinct executions**, with 375 repeated slots; this is not a count of independent requirements or statistically independent observations. Coverage is reported per version because requirements apply per version, and an untested requirement scores zero rather than being excluded (Table 2).

Table 2

Demonstrated coverage of registered information requirements. Preservation in both profiles and structure in the expanded profile have identical counts (third column). Percentages use all *N* applicable requirements, including unassessed ones. ND denotes tested but not demonstrated; U denotes unassessed. No requirement was partial in any version/profile/axis.

VCF	<i>N</i>	Preservation (both); structure (expanded)	Structure (condensed)	ND ^a	U ^b
4.1	69	32 (46.4%)	26 (37.7%)	6	37
4.2	72	34 (47.2%)	28 (38.9%)	6	38
4.3	74	35 (47.3%)	29 (39.2%)	6	39
4.4	83	38 (45.8%)	31 (37.3%)	7	45
4.5	91	51 (56.0%)	37 (40.7%)	14	40

^a Condensed structure only; all other profile/axis combinations had zero ND requirements. ^b The same unassessed count applies to each profile and axis within a version.

All 210 preservation tests passed in each profile, as did all 210 expanded-structure tests. In the condensed profile, 165 structure tests passed and 45 failed; every failure concerned per-sample FORMAT access that the profile stores in encoded vectors rather than individual sample resources. In VCF 4.5 specifically, 18 failed condensed-structure tests affected 14 requirements: failed tests and affected requirements are different counts. Every passing query slot rejected the empty-graph control and changed its answer under at least one referenced-predicate deletion. Thus the observed structural difference was a sample-profile limitation under the no-decoding axis, not evidence that VCF Core could not express those values structurally: the corresponding expanded tests passed.

The preservation and expanded-structure scores ranged from 45.8% to 56.0%, while condensed-structure scores ranged from 37.3% to 40.7%. The lower percentages than in the curated inventory reflect a different, source-derived denominator and an executable-evidence requirement, not a measured loss of represented content. For VCF 4.5, the 40 unassessed requirements comprised 37 awaiting the planned version-specific fixtures and queries, the reference-block requirement R45 whose tests were withdrawn because the source interpretation was disputed at review, and two broad requirements for the meanings and values of all reserved INFO and FORMAT keys (R67/R68). Passing selected field examples and matching all extracted Number/Type rows did not discharge those two broad requirements. The supported RQ1 result is therefore full representation of the curated list and demonstrated retrieval of a bounded subset of the registered information requirements, with the remaining test gaps explicit; neither assessment establishes complete specification coverage. Table 3 places these results alongside the separate implementation cross-checks.

Table 3

Reported evidence, its denominator and the claim it supports. The denominators count different things and must not be combined into a single percentage.

Assessment	Result	Supported interpretation
Curated VCF 4.5 inventory	104/104	All inventoried logical constructs have preserved, structured representations.
Enforcement within that inventory	87/104	A stated requirement is checked for these constructs.
Reserved Number/Type rows	333/333	Extracted declarations match the version-specific artifacts.
Demonstrated preservation, by version	32/69, 34/72, 35/74, 38/83, 51/91	Both profiles; expanded structure has the same counts. Condensed structure is reported separately in Table 2.
Distinct executed checks	465	Non-repeated query/answer/fixture checks within each case and profile, not independent requirements.
Cross-producer replay	693/840	Checks on which the vocabulary's materializer and the cited converter agree and both pass.
Semantic round-trip	109/109 records, 151/151 genotypes, 431/431 header lines, 400/401 field values	Fixed columns, sample genotypes, header lines, INFO entries and FORMAT subfields recovered from structured properties alone.
Recorded full validation run	21 fixtures	These examples pass the implemented validation layers.

A previous version of this assessment reported 491 reviewed requirements of which 133 had complete evidence. That generation has been superseded: requirements are now scored per version and per profile, and no single figure corresponds to the earlier 133.

3.3. Whether the claimed coverage survives a second producer

The assessment above builds its witness graphs with the vocabulary repository's own materializer, whereas the implementation this paper cites is a separate program, and nothing had established that a query and expected answer written against one also holds over the other — the gap between “the

vocabulary can express this” and “the software people will use does”. The two programs share an author but no code: one materializes graphs from a Python fixture builder, the other from RML mappings and streaming emitters. Independence here therefore means independent implementations, not independent authorship, and the check is correspondingly weaker than third-party reproduction.

We replayed the existing queries, against the existing reviewed expected answers, over graphs produced by VCF-RDFizer. Every fixture the assessment uses is converted in both sample profiles — all 38 of them, not a selection — so each of the 53 requirements that has a test is exercised against both producers. That gives 840 checks — the same 840 query executions the assessment itself performs, now performed twice over, once per producer. Of them, **693 pass for both producers** and **45 fail for both**. The 45 are all condensed-profile structure queries, the storage trade-off Section 2.2 describes: both producers answer them the same way, and that way is “no”.

On 102 checks, spanning 15 requirements, the materializer answers and the converter does not. No check goes the other way. Because the paper’s claim is about the vocabulary rather than about one program, what matters is why the converter is silent, so we examined every one.

Table 4

Why the converter fails a check the materializer passes. Every requirement is accounted for; the right-hand column is what would have to change to close it.

Cause	Reqs.	What is missing
A declared term the converter does not emit	12	The query asks for a property the vocabulary defines and the converter never writes, so the query matches nothing. Twenty such properties, covering percent-encoded text, padding, gVCF reference blocks, breakend mates, sample-level filter codes, per-allele phase sets, base-modification strand and allele indices, and the per-item layer for fixed-size value lists.
A decomposition that stops early	1	Tandem repeats: the summary row is emitted, the per-allele components are not.
A datatype the vocabulary leaves open	1	QUAL. The materializer writes <code>vcfc:VCFFloat</code> , the converter <code>xsd:decimal</code> ; both satisfy the shape, and only a query projecting <code>DATATYPE()</code> can tell them apart.
An encoding convention	1	An empty local-allele list, written <code>"</code> by one producer and <code>". "</code> by the other.

Table 4 gives the result that matters: **none of the 15 is a case of the vocabulary being unable to express something**. All twenty of the absent properties are declared in the cited release; the converter simply does not write them yet. The remaining three are a partial decomposition, a datatype the vocabulary deliberately permits two ways, and a difference in how an empty list is spelled. So across all 840 checks, zero divergences trace to a representation gap — but twelve of the fifteen do trace to work the converter has not done, which bounds what an implementation claim can say.

This check earned its keep during development. An earlier converter bound `Number=LA` and `Number=LR` items to allele indices positionally, ignoring the sample’s LAA subset: a depth list of 20, 30, 10 under `LAA=2, 4` landed on alleles 0, 1 and 2 instead of 0, 2 and 4 — plausible, well-formed values on the wrong alleles, which no existing test caught because every existing test ran against the other producer. Release 3.0.3 resolves local alleles correctly, and the requirements exercising them pass for both producers here.

A complementary check recovers logical content rather than comparing query answers. Across the same 38 fixtures we reconstruct CHROM, POS, ID, REF, ALT, QUAL, FILTER and the GT subfield from the RDF alone: REF and ALT from ordered `alleleIndex/alleleValue` resources, GT from ordered `GenotypeAlleleCall` resources with their phasing status, so no untouched source string serves as a shortcut. All 109 records are recovered, together with all 151 sample genotypes. A companion check does the same for the rest of the file: all **431 header lines**, recovered by source line number and compared on key and value, and every INFO entry and non-GT FORMAT subfield, rebuilt from ordered value items rather than from a preserved column, of which **400 of 401** recover. The exception is an

empty Number=LA list, where the converter emits the field as a resource but gives it no value; that is requirement R27 above, reached by a second route.

Three bounds, all by design. INFO and the non-GT FORMAT subfields are out of scope. The check runs on expanded-profile graphs only, since decoding a condensed vector needs the accessor requirement R21 identifies as missing. And one normalisation is applied to the genotype: VCF 4.4 onwards permits a leading phase indicator, so that | 0 | 1 and 0 | 1 describe the same phasing, and the vocabulary records that difference as lexical rather than semantic — phaseIndicator carries the effective first indicator, and only the preserved source string says whether it was written out. Reproducing that character would therefore require reading the very string this check avoids, so a genotype differing only by it counts as recovered; allele order, missing calls, ploidy and every indicator between a pair must match exactly. This is not whole-file reconstruction.

Where the two sample profiles agree. Because every case runs in both profiles, the profile claim can be stated precisely rather than asserted. The profiles return the same outcome on 352 of 420 comparable checks. The 68 disagreements fall on 15 requirements, and they share one property: every one reads data held inside a sample — either a FORMAT value or a genotype. Questions about the file, its header, a record or an allele agree on every check in both profiles.

That is the trade-off working as designed rather than a defect. The condensed profile stores sample values as vectors instead of as one resource per sample and field, so a query written to walk per-sample resources finds nothing there; recovering the same value requires decoding the vector, which these queries deliberately do not do. The supported claim is therefore narrower than “the profiles are equivalent”: they agree for file, header, record and allele questions, and diverge by design wherever a query descends into a sample. Three of the 15 concern local alleles and base modifications, so the features this paper highlights are among those a condensed graph answers less completely.

Answers to the three questions. RQ1 is answered for the 53 of 94 requirements with at least one executed case, the great majority passing. RQ2 is answered for 17 of the 36 version-conditioned requirements, with the profiles behaving as documented: equivalent above the sample level, divergent below it. RQ3 is answered for 27 of the 38 requirements on the provenance chain an integration query must traverse, which Section 3.5 then exercises end to end. None of these fractions is a conformance claim; each is the share of requirements whose behaviour has been checked at all.

Remaining gaps, classified. Of the requirements exercised, none reveals a representation gap. One correctness defect was found and fixed; three requirements await structured accessors in the converter; one reflects an interpretation the vocabulary deliberately permits both ways; one is an encoding convention. The largest limitation is absence of evidence rather than negative evidence: 41 of the 94 requirements have no test case yet, concentrated in structural-variant fields with version-specific semantics. Allele-dependent values, this paper’s central modelling claim, are tested for 4 of the 11 requirements that bear on them — adequate to demonstrate the mechanism and to catch the defect above, not adequate to claim general correctness. Outside that classification, four validator findings remain from the release’s own conformance checks: custom structured-header IDs can be absent or duplicated without rejection; numeric base-modification keys can be accepted with incorrect Number/Type pairs; a partially missing phase-set-list example is rejected under an unresolved interpretation; and mixed LF/CRLF line endings are rejected under an unconfirmed reading of the specification. Byte-level checking is reported separately, and neither BCF binary layout nor byte-for-byte reconstruction is a target of the logical model.

3.4. Implementation and reproducibility

The release provides OWL/Turtle modules, SHACL profiles, examples, versioned registries, SSSOM mappings and documentation [20]. VCF-RDFizer supplies a separate implementation: release 3.0.3 constructs file and record context with RML mappings and produces expanded or condensed sample

structures through streaming emitters [25]. The Section 3.2 assessment was produced with VCF Core 2.1.2 and the vocabulary’s own materializer, with no converter involvement; all converter-dependent results (Sections 3.3–3.5) used VCF-RDFizer 3.0.3, the release that carries the corrections the cross-producer check prompted. Both are tagged, so every figure below comes from a released artifact rather than a working branch. Converting the Section 2.2 example emits 242 expanded and 168 condensed triples, drawing on 93 and 80 distinct VCF Core terms respectively. Every one of those terms is declared in VCF Core 2.1.2, and both graphs conform, with no violations, to the shared and the VCF 4.5 SHACL profile when the shapes are loaded with the vocabulary and RDFS inference, as the vocabulary’s own validation does. The depth query in Listing ?? returns its expected single row against the expanded graph. The converter pins no vocabulary version, so this relationship is established by the behaviour recorded here rather than by a declaration, and it is evidence for the terms these graphs exercise, not for all 467 declared terms or for every historical VCF version.

Every figure in this section comes from one execution of a single script, which fetches the vocabulary and both converter releases at pinned revisions, pulls their container images by digest, converts the fixtures, runs each analysis, and records the machine, the toolchain versions, and every command with its exit code and duration alongside the results. One limitation remains: the toolchain inside the image is not independently pinned, so the image digest, not a list of component versions, is what fixes it. The counts above are converter output; the hand-authored illustration file in the release is smaller, because it omits the allele resources and per-sample genotype and allele-call resources that the converter materializes.

The recorded validation run accepts 21 fixtures and includes a negative control, alongside a regression suite of 101 tests. The published workflow recomputes the reported counts and checks evidence references against the maintained artifacts, and each result in Section 3.2 has a script that reproduces it from the pinned inputs. Review of coverage judgements and mappings nonetheless remains a human task. The established result is full representation of the curated VCF 4.5 inventory with version-aware support across VCF 4.1–4.5, demonstrated to be reproducible with an independent converter on the subset tested — not complete conformance.

3.5. One integration example, with checked answers

The claims above concern representation and query behaviour within the VCF world. To test whether the retained context survives contact with an external annotation, we ran one small end-to-end example and checked its answers against expectations authored in advance.

The example is bounded to a single difficulty: allele-dependent values under VCF 4.5 local alleles. LAD is declared `Number=LR`, meaning one value per allele of *the sample’s own* local allele subset named by LAA, not per entry of the record’s ALT column. Answering “how deep is this alteration in this sample?” therefore requires the declaration, the sample’s LAA and the allele’s ordinal simultaneously. The input is a pinned four-site, single-sample VCF 4.5 fixture, each site written twice, once with local alleles and once globally; the annotations are a local, explicitly synthetic snapshot of five flagged alterations, labelled as such in the file, and local rather than federated so the example returns the same answer on every run. The link is made in the query rather than by pre-minted identifiers: an annotation names an alteration by contig, position, reference and alternate allele, and the query joins on those four literals. Three assumptions are recorded because they are what would break the link — a shared assembly, as-written allele identity with no normalization or VRS matching, and that a VCF sample column is a sample and not a patient.

Asking which flagged alterations have supporting read depth of at least 20 returns two rows, identical to the expected answers: `chr1:1 G>C` at depth 30 and `chr1:2 A>T` at depth 25, each traced to its file, record, sample and the FORMAT declaration that gives the field its cardinality. The three flagged alterations absent from the answer are each absent for a stated reason rather than by oversight.

The example is worth reporting because of what a wrong implementation would do with it. Reading `Number=LR` positionally — taking the *n*th LAD value as belonging to the *n*th ALT allele, rather than to the *n*th member of the sample’s own LAA subset — returns *two rows again*, but `chr1:1 G>A` at depth

30 and chr1:2 A>C at 25. Those two alterations and those two depths are named in the expected answers as the positional-reading outcome, derived from the fixture text before any query ran. A check comparing result counts cannot tell the two apart. This is the concrete form of the paper’s argument: retaining the declaration, the local allele set and the allele ordinal is not bookkeeping, because dropping it yields a plausible, well-formed, wrong answer delivered with complete provenance.

What this example demonstrates is that the representation carries the interpretation context, that a mapping rule connects it to external annotation, and that the answers match those authored beforehand. What it does not demonstrate is equally clear: the input is synthetic and tiny, the annotations are invented and carry no biological or clinical meaning, the link assumes as-written allele identity, and a single fixture establishes correct local-allele handling here rather than in general.

4. Discussion

4.1. Connecting source observations to biomedical knowledge

VCF Core can make more of the evidence already present in a VCF file available to semantic applications. It retains observations and their interpretation context, while other models describe specimens, patients, diseases, treatments and biological variation. These layers can develop independently when their relationships are explicit.

Two laboratories reporting the same alteration in separate files could, after reference-aware normalization, link to a common variation description while each retains its own quality values, sample identities and provenance — so a disease association attaches to the description once rather than being copied into every source record, and a revised interpretation can be retrieved alongside the original observations. PROV, FALDO and the published alignments are starting points; the normalization and instance mappings themselves remain to be implemented and evaluated.

Sample identity needs the same separation. In a tumour/normal analysis two columns may be different specimens from one patient, and treating column names as patient identifiers destroys that distinction. An integration layer can link each `VCFSample` to its specimen and both specimens to the patient through a biomedical model such as HERO-Genomics [14]; VCF Core preserves the source side without replacing the clinical one.

4.2. A shared target for conversion engines

A reusable vocabulary also separates conversion strategy from representation decisions. A clinical site converting one sample for inspection and a research group converting a cohort can choose different sample profiles — expanded values for direct annotation, condensed vectors with selective expansion for scale — while retaining the same file, header and record model rather than growing incompatible vocabularies per workload. A pipeline spanning VCF 4.2 and 4.5 data can likewise select the registry and validation overlay from `fileFormat` instead of silently imposing the newest cardinality on an older declaration.

That suggests an evaluation order for future work: compare recovered source values and query answers first, as Section 3.3 does, and only then measure conversion time, peak memory, serialized size and loading cost. Section 2.2 is explicit that the profiles trade query granularity rather than demonstrated storage savings, and the present results do not establish which implementation is most efficient.

4.3. SPARQL questions that combine genomic and clinical evidence

Section 3.5 established the pattern at the smallest scale that requires it: an external annotation joined to sample-level evidence, with the answer depending on interpretation context that the representation retains. The questions below extend that pattern to clinical settings. They are proposed research scenarios and *not* evaluated workflows: unlike the example in Section 3.5, none has been executed, and each requires additional biomedical data, reviewed instance mappings and appropriate access controls.

SPARQL provides the query structure; it does not supply missing interpretation or establish that a returned result is clinically actionable.

Three questions. A *pharmacogenomic* review asks which patients with an active clopidogrel prescription have an externally established CYP2C19 phenotype relevant to a guideline for their indication, and which source observations support that assessment; the CPIC guideline shows why phenotype and indication both matter [27], and a workflow such as PharmCAT separates variant processing, allele matching and clinical interpretation [28] — steps that must not collapse into reading one genotype token as a metabolizer phenotype. A *rare-disease* investigation asks which candidate alterations lie in genes associated with diseases sharing the patient’s recorded phenotypic features, joining specimen-linked calls to normalized alterations, gene associations and phenotype descriptions such as HPO [29]; its governing constraint is that a row absent from a variant-only VCF is not evidence that a parent is homozygous reference. An *oncology* review asks which alterations detected in a tumour have curated therapeutic evidence for the patient’s cancer type, joining a resource such as CIViC [30] to normalized alterations and retrieving tumour and normal calls through their specimen links; a missing normal call likewise remains missing evidence, not proof of tumour specificity.

Each reaches its sample-level evidence through the same declaration-to-value path that Section 3.5 executes, and each needs clinical assertions from an application schema that is not part of VCF Core. The vocabulary retains the observations; completeness, segregation and therapeutic interpretation require evidence it does not carry.

These scenarios could be implemented over a materialized graph or, where source interfaces and authorization permit, through federated SPARQL as demonstrated in related work [31]. Availability of RDF and vocabulary alignments alone does not guarantee complete or performant answers. Future experiments should evaluate answer correctness, instance-link accuracy, provenance recovery and source availability before making claims about medical utility or query speed. The opportunity is to investigate these questions against a common representation instead of repeatedly reconstructing VCF semantics within each application.

5. Conclusion

VCF Core provides a shared semantic foundation for representing VCF observations together with the context needed to interpret them. It combines reuse of existing models, explicit source structure, alternative sample representations and version-specific artifacts for VCF 4.1–4.5. All 104 constructs in the curated VCF 4.5 inventory are represented, and the claims that matter have been executed rather than asserted: the specification-derived checks were replayed in full across a second producer, 693 of 840 passing for both with no divergence traceable to the vocabulary, all 109 records and 151 sample genotypes round-trip through structured properties, and one integration example returns answers authored in advance together with their source provenance.

The remaining work is specific rather than general. Evidence is absent for 41 of the 94 requirements, concentrated in structural-variant fields, and thinner than we would like for allele-dependent values in particular; three structured accessors are not yet implemented in the cited converter; one QUAL datatype question is an interpretation the vocabulary deliberately leaves open; and the per-sample condensed profile cannot yet answer the hardest missingness question. None of these is a representation gap, and none is hypothetical — each is a named requirement with a recorded result.

What this foundation makes practical is the step from representing genomic files to connecting their contents with biomedical knowledge. Conversion engines can target the same model, integration layers can preserve the distinction between observations and interpretations, and SPARQL studies can examine medically relevant questions with traceable source evidence. The clinical scenarios in Section 4.3 remain future work: the VCF-side representation they would build on is in place and checked, while the biomedical models and reviewed instance mappings they equally require are not.

Acknowledgments

Project funding provided from the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N). Ruben Taelman is a postdoctoral fellow of the Research Foundation – Flanders (FWO) (1202124N).

Declaration on Generative AI

OpenAI Codex and Claude assisted with preparatory work on the vocabulary, assessment artifacts and manuscript. ChatGPT assisted with restructuring, prose revision, reference checks and prospective query examples.

References

- [1] C. Gaudet-Blavignac, J. L. Raisaro, V. Touré, S. Österle, K. Cramer, C. Lovis, A national, semantic-driven, three-pillar strategy to enable health data secondary usage interoperability for research within the Swiss Personalized Health Network: Methodological study, *JMIR Medical Informatics* 9 (2021) e27591. URL: <https://medinform.jmir.org/2021/6/e27591/>. doi:10.2196/27591.
- [2] T. E. Putman, K. Schaper, N. Matentzoglou, V. P. Rubinetti, F. S. Alquaddoomi, C. Cox, J. H. Caufield, G. Elsarboukh, S. Gehrke, H. Hegde, J. T. Reese, I. Braun, R. M. Bruskewich, L. Cappelletti, S. Carbon, A. R. Caron, L. E. Chan, C. G. Chute, K. G. Cortes, V. De Souza, T. Fontana, N. L. Harris, E. L. Hartley, E. Hurwitz, J. O. B. Jacobsen, M. Krishnamurthy, B. J. Laraway, J. A. McLaughlin, J. A. McMurry, S. A. T. Moxon, K. R. Mullen, S. T. O’Neil, K. A. Shefchek, R. Stefancsik, S. Toro, N. A. Vasilevsky, R. L. Walls, P. L. Whetzel, D. Osumi-Sutherland, D. Smedley, P. N. Robinson, C. J. Mungall, M. A. Haendel, M. C. Munoz-Torres, The Monarch Initiative in 2024: an analytic platform integrating phenotypes, genes and diseases across species, *Nucleic Acids Research* 52 (2024) D938–D949. URL: <https://academic.oup.com/nar/article/52/D1/D938/7449493>. doi:10.1093/nar/gkad1082.
- [3] B. Smith, M. Ashburner, C. Rosse, J. Bard, W. Bug, W. Ceusters, L. J. Goldberg, K. Eilbeck, A. Ireland, C. J. Mungall, The OBI Consortium, N. Leontis, P. Rocca-Serra, A. Ruttenberg, S.-A. Sansone, R. H. Scheuermann, N. Shah, P. L. Whetzel, S. Lewis, The OBO Foundry: coordinated evolution of ontologies to support biomedical data integration, *Nature Biotechnology* 25 (2007) 1251–1255. URL: <https://www.nature.com/articles/nbt1346>. doi:10.1038/nbt1346.
- [4] F. Belleau, M.-A. Nolin, N. Tourigny, P. Rigault, J. Morissette, Bio2RDF: Towards a mashup to build bioinformatics knowledge systems, *Journal of Biomedical Informatics* 41 (2008) 706–716. URL: <https://www.sciencedirect.com/science/article/pii/S1532046408000415>. doi:10.1016/j.jbi.2008.03.004.
- [5] P. Danecek, A. Auton, G. Abecasis, C. A. Albers, E. Banks, M. A. DePristo, R. E. Handsaker, G. Lunter, G. T. Marth, S. T. Sherry, G. McVean, R. Durbin, 1000 Genomes Project Analysis Group, The variant call format and VCFtools, *Bioinformatics* 27 (2011) 2156–2158. URL: <https://academic.oup.com/bioinformatics/article/27/15/2156/402296>. doi:10.1093/bioinformatics/btr330.
- [6] VCF, The Variant Call Format Specification: VCFv4.5 and BCFv2.2, 2026. URL: <https://samtools.github.io/hts-specs/VCFv4.5.pdf>, 25 February 2026 printing, version e821e4f; accessed 6 September 2026.
- [7] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. G. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. C. ’t Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, B. Mons, The FAIR guiding principles for scientific data management and stewardship, *Scientific Data* 3 (2016) 160018. URL: <https://www.nature.com/articles/sdata201618>. doi:10.1038/sdata.2016.18.

- [8] C. Bahim, C. Casorrán-Amilburu, M. Dekkers, E. Herczog, N. Loozen, K. Repanas, K. Russell, S. Stall, The FAIR Data Maturity Model: An approach to harmonise FAIR assessments, *Data Science Journal* 19 (2020) 41. URL: <https://datascience.codata.org/articles/dsj-2020-041>. doi:10.5334/dsj-2020-041.
- [9] S. Decker, S. Melnik, F. van Harmelen, D. Fensel, M. Klein, J. Broekstra, M. Erdmann, I. Horrocks, The semantic web: the roles of XML and RDF, *IEEE Internet Computing* 4 (2000) 63–73. URL: <https://ieeexplore.ieee.org/document/877487/>. doi:10.1109/4236.877487.
- [10] C. Bizer, T. Heath, T. Berners-Lee, Linked Data—the story so far, *International Journal on Semantic Web and Information Systems* 5 (2009) 1–22. URL: <https://doi.org/10.4018/jswis.2009081901>. doi:10.4018/jswis.2009081901.
- [11] I. M. Lubin, N. Aziz, L. J. Babb, D. Ballinger, H. Bisht, D. M. Church, S. Cordes, K. Eilbeck, F. Hyland, L. Kalman, M. Landrum, E. R. Lockhart, D. Maglott, G. Marth, J. D. Pfeifer, H. L. Rehm, S. Roy, Z. Tezak, R. Truty, M. Ullman-Cullere, K. V. Voelkerding, E. A. Worthey, A. W. Zaranek, J. M. Zook, Principles and recommendations for standardizing the use of the next-generation sequencing variant file in clinical settings, *The Journal of Molecular Diagnostics* 19 (2017) 417–426. URL: <https://doi.org/10.1016/j.jmoldx.2016.12.001>. doi:10.1016/j.jmoldx.2016.12.001.
- [12] J. Baran, B. S. B. Durgahee, K. Eilbeck, E. Antezana, R. Hoehndorf, M. Dumontier, GFVO: the Genomic Feature and Variation Ontology, *PeerJ* 3 (2015) e933. URL: <https://peerj.com/articles/933/>. doi:10.7717/peerj.933.
- [13] E. D. S. Penha, E. Iriabho, A. Dussaq, D. M. de Oliveira, J. S. Almeida, Isomorphic semantic mapping of variant call format (VCF2RDF), *Bioinformatics* 33 (2017) 547–548. URL: <https://academic.oup.com/bioinformatics/article/33/4/547/2593587>. doi:10.1093/bioinformatics/btw652.
- [14] M. Cazzaro, I. G. Gut, L. Menotti, M. Rueda, G. Silvello, HERO-Genomics: An ontology for integration and access of multicenter genomic data, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 33–42. URL: https://ceur-ws.org/Vol-4196/paper_17.pdf.
- [15] S. Kawashima, T. Fujisawa, T. Katayama, Genome variation ontology for annotation of complex structural variations, in: *Proceedings of the 14th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2023)*, volume 3415 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2023, pp. 126–127. URL: <https://ceur-ws.org/Vol-3415/paper-22.pdf>.
- [16] A. H. Wagner, L. Babb, G. Alterovitz, M. Baudis, M. Brush, D. L. Cameron, M. Cline, M. Griffith, O. L. Griffith, S. E. Hunt, D. Kreda, J. M. Lee, S. Li, J. Lopez, E. Moyer, T. Nelson, R. Y. Patel, K. Riehle, P. N. Robinson, S. Rynearson, H. Schuilenburg, K. Tsukanov, B. Walsh, M. Konopko, H. L. Rehm, A. D. Yates, R. R. Freimuth, R. K. Hart, The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification, *Cell Genomics* 1 (2021) 100027. doi:10.1016/j.xgen.2021.100027.
- [17] Global Alliance for Genomics and Health, A federated ecosystem for sharing genomic, clinical data, *Science* 352 (2016) 1278–1280. doi:10.1126/science.aaf6162.
- [18] Global Alliance for Genomics and Health, HTS Specifications: SAM/BAM and Related High-Throughput Sequencing File Formats, <https://github.com/samtools/hts-specs>, ??? Repository containing the Variant Call Format specifications, including VCF versions 4.1–4.5.
- [19] SAM/BAM Format Specification Working Group, HTS format specifications: VCF specifications and earlier versions, Official specification index, 2026. URL: <https://samtools.github.io/hts-specs/>, accessed 9 September 2026.
- [20] E. Crum, VCF Core Vocabulary, Version 2.1.2. GitHub repository, 2026. URL: <https://github.com/ecrum19/vcf-core-vocabulary/tree/3f06d18>, version 2.1.2 with the SSSOM alignment set; source snapshot of 14 September 2026.

- [21] E. Crum, R. Taelman, B. Buelens, G. Ertaylan, R. Verborgh, Semantifying genomic variant data: VCF to RDF conversion framework, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 199–200. URL: https://ceur-ws.org/Vol-4196/paper_6.pdf.
- [22] World Wide Web Consortium, PROV-O: The PROV Ontology, W3C Recommendation, World Wide Web Consortium, 2013. URL: <https://www.w3.org/TR/2013/REC-prov-o-20130430/>.
- [23] K. Eilbeck, S. E. Lewis, C. J. Mungall, M. Yandell, L. Stein, R. Durbin, M. Ashburner, The sequence ontology: a tool for the unification of genome annotations, *Genome Biology* 6 (2005) R44. URL: <https://link.springer.com/article/10.1186/gb-2005-6-5-r44>. doi:10.1186/gb-2005-6-5-r44.
- [24] J. T. Bolleman, C. J. Mungall, F. Strozzi, J. Baran, M. Dumontier, R. J. P. Bonnal, R. Buels, R. Hoehndorf, T. Fujisawa, T. Katayama, P. J. A. Cock, FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation, *Journal of Biomedical Semantics* 7 (2016) 39. URL: <https://link.springer.com/article/10.1186/s13326-016-0067-z>. doi:10.1186/s13326-016-0067-z.
- [25] VCF-RDFizer maintainers, VCF-RDFizer, Version 3.0.3. Open-source software, 2026. URL: <https://github.com/ecrum19/VCF-RDFizer/tree/be658a2>, tag v3.0.3; converted and validated 14 September 2026.
- [26] World Wide Web Consortium, Shapes Constraint Language (SHACL), W3C Recommendation, World Wide Web Consortium, 2017. URL: <https://www.w3.org/TR/2017/REC-shacl-20170720/>.
- [27] C. R. Lee, J. A. Luzum, K. Sangkuhl, R. S. Gammal, M. S. Sabatine, C. M. Stein, D. F. Kisor, N. A. Limdi, Y. M. Lee, S. A. Scott, J.-S. Hulot, D. M. Roden, A. Gaedigk, K. E. Caudle, T. E. Klein, J. A. Johnson, A. R. Shuldiner, Clinical pharmacogenetics implementation consortium guideline for CYP2C19 genotype and clopidogrel therapy: 2022 update, *Clinical Pharmacology & Therapeutics* 112 (2022) 959–967. doi:10.1002/cpt.2526.
- [28] PharmCAT contributors, PharmCAT methods, Official documentation, 2026. URL: <https://pharmcat.org/methods/>, accessed 9 September 2026.
- [29] Human Phenotype Ontology Consortium, The human phenotype ontology, Project website and documentation, 2026. URL: <https://hpo.jax.org/>, accessed 9 September 2026.
- [30] M. Griffith, N. C. Spies, K. Krysiak, J. F. McMichael, A. C. Coffman, A. M. Danos, B. J. Ainscough, C. A. Ramirez, D. T. Rieke, L. Kujan, E. K. Barnell, A. H. Wagner, Z. L. Skidmore, A. Wollam, C. J. Liu, M. R. Jones, R. L. Bilski, R. Lesurf, Y.-Y. Feng, N. M. Shah, M. Bonakdar, L. Trani, M. Matlock, A. Ramu, K. M. Campbell, G. C. Spies, A. P. Graubert, K. Gangavarapu, J. M. Eldred, D. E. Larson, J. R. Walker, B. M. Good, C. Wu, A. I. Su, R. Dienstmann, A. A. Margolin, D. Tamborero, N. Lopez-Bigas, S. J. M. Jones, R. Bose, D. H. Spencer, L. D. Wartman, R. K. Wilson, E. R. Mardis, O. L. Griffith, CIViC is a community knowledgebase for expert crowdsourcing the clinical interpretation of variants in cancer, *Nature Genetics* 49 (2017) 170–174. doi:10.1038/ng.3774.
- [31] A. Bodrug-Schepers, H. Chabane, G. Montoya, P. Serrano-Alvarado, R. Redon, A. Gaignard, Semantic beacons: a framework to support federated querying over genomic variants and public knowledge graphs, in: *Proceedings of the 16th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025, pp. 21–32. URL: https://ceur-ws.org/Vol-4196/paper_16.pdf.